

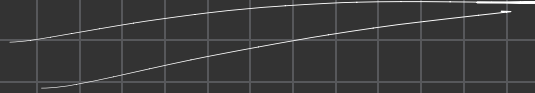
Telegram



Discord



Closures



```
function outter(){
  let a = 5;
  function inner(){
    console.log(a);
    let a = 7;
  }
  inner()
}
outter()
```

creation Exe
a: 5
inner()

```
function outter(){
  const a = 5;
  function inner(){
    console.log(a);
    var a = 7;
  }
  inner()
}
outter()
```

cre Exe
a: 7

Td2

Reference error

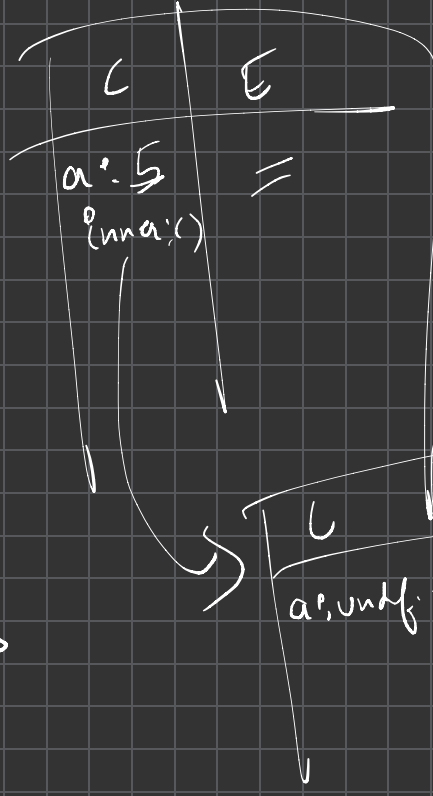
```

function outter() {
  const a = 5;
  function inner() {
    console.log(a);
    var a = 7;
  }
  inner()
}
outter()

```

undefined

scope



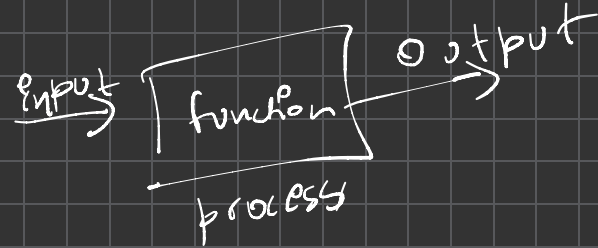
```
function outter(){
  const a = 5;
  function inner(){
    console.log(a);
  }

  return inner;
}
const response = outter();
```

parcel

printer

return 5 | 'hi' | variable |
 = true | 2 | 3 | 5
 keyword
 ↓
 return
 undefined | null
function



```
function outter(){  
  const a = 5;  
  function inner(){  
    console.log(a);  
  }  
  
  return inner;  
}  
  
const response = outter();
```

return 4th

function call

inner()
execute

inner fun

variable
debarita

Closure

↳ A function that remembers its
outer/parent scope variables.
└──────────────────────────┘
lexical environment

```
fun1() {  
  var a = 2  
  let b = 'Hi'  
  fun2() {  
  }  
}
```

fun2

- ↳ remembers its
parent scope
variable
- ↳ lexical
environment

```
outer() {  
  let a = 5
```

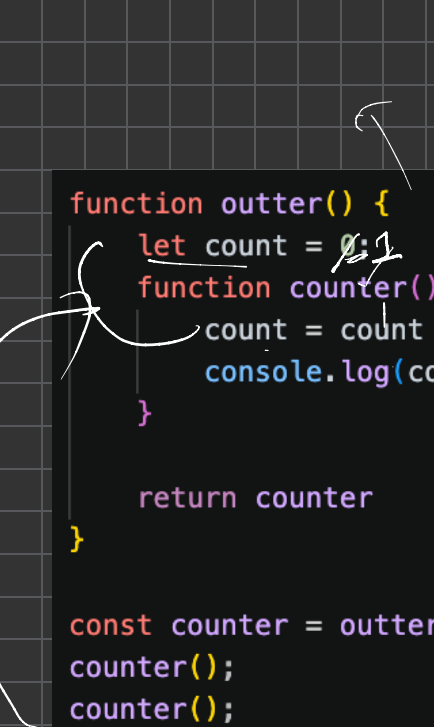
```
  inner() {  
    (lg(a))
```

```
  }
```

```
}
```

Reference

150
a



```
function outter() {  
  let count = 0;  
  function counter() {  
    count = count + 1;  
    console.log(count);  
  }  
  
  return counter  
}  
  
const counter = outter();  
counter();  
counter();
```

```

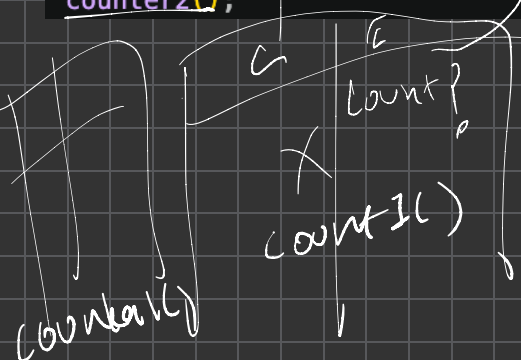
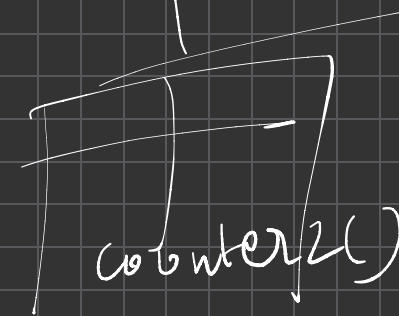
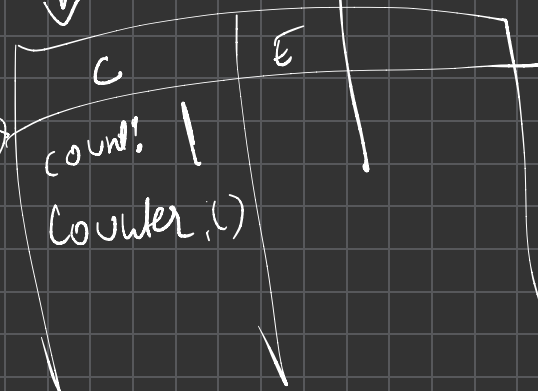
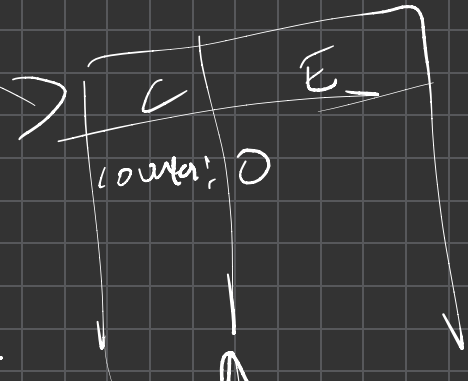
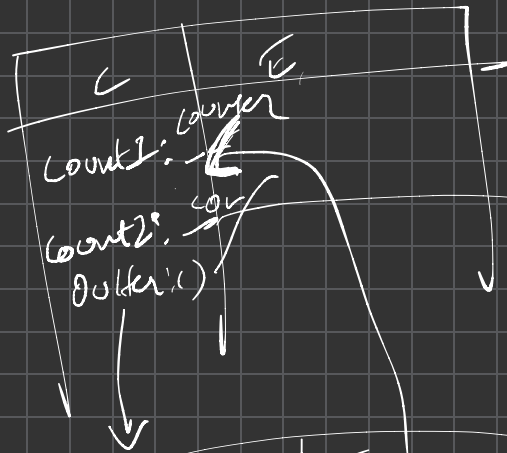
function outter() {
  let count = 0;
  function counter() {
    count = count + 1;
    console.log(count);
  }
  return counter
}

```

```

const counter1 = outter();
const counter2 = outter();
counter1();
counter1();
counter2();

```



closure

Projects

4

5

→ Basic JS ✓

→ Frontend Related JS

→ Advance JS

Revision

Mutability

↳ change kar sakte ho (original value bhi change ho jayegi)

Immutability

↳ original value change nhi hoti

